Re-Creating Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding

Maxwell Heefner

https://github.com/maxheef/ECE209_Project1.git

Task A

Reproduction target is Table 1 in the VCD paper, specifically the MSCOCO (POPE) + LLaVA-1.5 results. In this project, you will (i) reproduce the Table 1 setting on two POPE splits (random and popular), comparing Regular decoding and VCD with the following evaluation settings:

- Dataset source: MSCOCO (POPE split) Val2014
- Model: LLaVA-1.5 7b
- Methods: Regular decoding (baseline) vs. VCD decoding
- POPE settings: random and popular
- VCD Parameters:
    - Seed: 55
    - Alpha: 1
    - Beta: 0.2
    - Noise Step: 500

Implementation

To re-create the results on POPE, the original Visual Contrastive Decoding (VCD) codebase was used as a baseline. A wrapper script was implemented to compare the performance on the POPE random and popular splits under both regular decoding and VCD.

Experiments were conducted on a Google Colab H100 GPU. The runtime environment needed to be re-configured to ensure compatibility with Python 3.9, Numpy 1.X, and PyTorch. To manage version control, the H100 pulled all relevant dependencies and code from GitHub and saved all metrics and variables locally.

The evaluation dataset consisted of the MSCOCO Val2014 split, and the model used was the LLaVA 1.5-7b. For consistency and to ensure POPE sensitivity to prompt phrasing, the message response template from the original project (*"Please answer this question with one word."*) remained unchanged. This will limit the model to answering with binary "yes" or "no". The desired parameters for $Seed, Alpha, Beta,$ and $Noise\ Step$ were taken from Leng et al. [1].

In order to calculate accuracy, precision, recall and F1 Score the True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN) responses were recorded through each iteration. To calculate the metrics seen in table 1, the following formulas were utilized:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 * Precision * \frac{Recall}{Precision + Recall}$$

$$Accuracy = \frac{TP + TN}{Total\ Questions}$$

$$Yes\ Proportion = \frac{Yes\ Answers}{Total\ Questions}$$

$$Unknown\ Prop = \frac{Unknown}{Total\ Questions}$$

All code is attached below, along with a README.md that explains how LLaVA 1.5-7b was evaluated on the MSCOCO Val2014 dataset using POPE random and popular splits, with both regular decoding and VCD. The attached code was used to generate the Table 1 results.

Dataset	**Setting**	**Model**	**Decoding**	**Accuracy**	**Precision**	**Recall**	**F1 Score**
MSCOCO	*Random*	LLaVA 1.5-7b	Regular	0.8293	0.9201	0.7213	0.8087
			VCD	**0.8553**	0.9369	0.762	**0.8404**
	Popular		Regular	0.8113	0.8797	0.7213	0.7927
			VCD	**0.836**	0.8944	0.762	**0.8229**

Table 1. Results on POPE. Regular decoding denotes direct sampling, whereas VCD refers to sampling from our proposed contrastive distribution p_{vcd}. The best performances within each setting are bolded[1].

Discussion

Both the random and popular splits showed increased accuracy when decoded with VCD. This is consistent with the original results reported by Leng et al [1]. VCD has consistently shown improvement from this experiment with the MSCOCO dataset for all metrics of accuracy, precision, recall and F1.

The largest improvements are seen in recall and F1, suggesting that VCD aids in reducing the number of missed TP. Meaning, that the model will be less likely to deny objects that are present in the image.

POPE is designed to evaluate object hallucinations by asking binary questions (e.g., "*Is there a bird in the image?*") that test susceptibility to the presence of language priors or biases. The difference in these two splits is demonstrated in how negative objects are sampled;

- Random Split: negative objects are sampled uniformly
- Popular Split: negative objects are sampled from more common categories (increasing chances of priors in hallucinations)

Since improvement was seen in both splits, it is implied that VCD aids in not only correctly determining if random objects are present, but also mitigates the effects of priors. Meaning that VCD can demonstrate improvement both visually and against language-based biases.

This improvement is limited to adding gaussian noise to each perturbation of the images in the dataset. However, Leng et al. [1] note that more robust techniques – such as object-level blurring – would result more effective hallucination mitigation. By reproducing the results from Leng et al. [1], it has been demonstrated that VCD shows improvements in mitigating object hallucination, but with more robust techniques, even further improvement should be expected.

References

[1] *Sicong Leng, Hang Zhang, Guanzheng Chen, Xin Li, Shijian Lu, Chunyan Miao, and Lidong Bing. Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding*

[2] *Bingqian Liu, Fu Zhang*, Guoqing Chen, and Jingwei Cheng. Multi-Frequency Contrastive Decoding: Alleviating Hallucinations for Large Vision-Language Models*

[3] *Jihoon Lee and Min Song. Retrieval Visual Contrastive Decoding to Mitigate Object Hallucinations in Large Vision-Language Models.*

maxheef / ECE209_Project1 Type [/] to search

Code Issues Pull requests Actions Projects Security Insights

ECE209_Project1 / README.md

Max and Max Midterm writeup included and ReadMe comments added

57d0807 · 3 minutes ago History

Preview Code Blame 80 lines (69 loc) · 3.6 KB Raw

Maxwell Heefner

ECE209AS Trustworthy AI

18 Feb 26

Project1 Task A

This folder contains a items for Task A that reuses code from `originalProject`

Files

- `project1.ipynb` : Jupyter Notebook that will run all code and configure the Colab H100 GPU for use and then re-create Table 1 from *Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding*
  - Note: All files that the H100 GPU will reference are cloned from GitHub into `/content` storage then writes the outputs to temporary storage on Colab

`/content/VCD_project/project1/output`

- `setup_h100_env.sh` : Creates Python 3.9 conda env and installs compatible deps for Colab H100 GPU.
- `run_table1.sh` : Runs 4 iterations (*random/popular* and *regular/vcd*) and re-creates Table 1 from: *Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding* by *Leng et al.*
  - This file accomplishes this by calling `/eval/object_hallucination_vqa_llava.py` from the original VCD project and implementing all of the desired parameters (*SEED, Alpha, Beta, Noise Step, etc.*).
  - POPE datasets were used from `originalProject` for both random and popular, located here: `experiments/data/POPE/coco/coco_pope_random.json` and `experiments/data/POPE/coco/coco_pope_popular.json`
  - In order to evaluate the results `/eval/eval_pope.py` from `originalProject` was utilized to keep track of True/False Positve/Negative results and calculate *Accuracy*, *Precision*, *Recall* and *F1 Score*

Using an H100 GPU thorugh Colab

- Multiple compatibility errors were noted when using the some of the `originalProject` files, because python 3.9 and numpy 1.x must be used

Summary of the commands

Clone GitHub repository to Colab GPU

```
%%bash
set -e
cd /content
if [ ! -d VCD_project/.git ]; then
  git clone --depth 1 https://github.com/maxheef/ECE209_Project1.git
fi
```

Configure the H100 GPU for python 3.9 and numpy 1.x

```
%%bash
set -e
cd /content/VCD_project
bash project1/setup_h100_env.sh /content/VCD_project
```

Download the MSCOCO imageset

```
%%bash
set -e
mkdir -p /content/datasets/coco
cd /content/datasets/coco
if [ ! -d val2014 ]; then
  wget -q http://images.cocodataset.org/zips/val2014.zip
  unzip -q val2014.zip
fi
```

Create Table 1 variables and output in neat format

```
%%bash
set -e
cd /content/VCD_project
PYBIN=$(cat /tmp/project1_python_bin.txt)
PYTHON_BIN="$PYBIN" bash project1/run_table1.sh \
  liuhaotian/llava-v1.5-7b \
  /content/datasets/coco/val2014 \
  55
```

Outputs:

- `/content/VCD_project/project1/output`

Documentaion statement

The papers located in `/Papers` were referenced to understand Mitigating Object Hallucinationw with VCD, in addition to the `https://github.com/DAMO-NLP-SG/VCD.git` repository which was cloned into `/originalProject` The cloned Git Repository was used to run the experiment, specifically with `/eval/object_hallucination_vqa_llava.py` In addition to the provided material and Git repository, the VSCode Extension of CODEX was utilized to aid with template generation and creation of `setup_h100_env.sh` to fix compatibility errors with the Colab H100 GPU. *Prettier* was used to help format and propse commands to generate clean and efficient code.

Citation

@article{damonlpsg2023vcd, author = {Sicong Leng, Hang Zhang, Guanzheng Chen, Xin Li, Shijian Lu, Chunyan Miao, Lidong Bing}, title = {Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding}, year = 2023, journal = {arXiv preprint arXiv:2311.16922}, url = {https://arxiv.org/abs/2311.16922} }

maxheef / ECE209_Project1 Type [/] to search

Code Issues Pull requests Actions Projects Security Insights

ECE209_Project1 / project1 / setup_h100_env.sh

Max and Max Documentation statement added c7860a8 · 4 days ago History

Code Blame Executable File · 52 lines (43 loc) · 1.72 KB Raw

```
# Maxwell Heefner
# ECE209AS Trustworthy AI
# 18 Feb 26
# setup_h100_env.sh
#!/usr/bin/env bash

# Documentation: CODEX Extension utilized to troubleshoot runtime errors for comp
# and aided in generation of this document to ensure a Colab H100 GPU had proper
set -euo pipefail

ROOT_DIR="${1:-/content/VCD_project}"
ORIG_DIR="$ROOT_DIR/originalProject"
ENV_NAME="${2:-vcd39}"

if [[ ! -d "$ORIG_DIR" ]]; then
  echo "Missing originalProject directory: $ORIG_DIR"
  exit 1
fi

if [[ ! -x /usr/local/miniconda/bin/conda ]]; then
  wget -q https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -
  bash /tmp/miniconda.sh -b -p /usr/local/miniconda
fi

source /usr/local/miniconda/etc/profile.d/conda.sh
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/mai
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r |
conda config --set always_yes yes --set changeps1 no

```

```
if ! conda env list | grep -q "^${ENV_NAME}[[:space:]]"; then
  conda create -n "$ENV_NAME" python=3.9
fi

conda activate "$ENV_NAME"
python -m pip install --upgrade pip

cd "$ORIG_DIR"
pip install -r requirements.txt
# H100 needs newer torch than repo pin.
pip install --upgrade --index-url https://download.pytorch.org/whl/cu121 torch==2
# Keep ABI compatibility with older compiled deps.
pip install --force-reinstall 'numpy<2'

echo "/usr/local/miniconda/envs/${ENV_NAME}/bin/python" > /tmp/project1_python_bi
python - <<'PY'
import torch, numpy
print('Torch:', torch.__version__)
print('NumPy:', numpy.__version__)
print('CUDA:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
PY
```

maxheef / ECE209_Project1 Type [/] to search

Code Issues Pull requests Actions Projects Security Insights

ECE209_Project1 / project1 / run_table1.sh

Max and Max updated comments 337f2a7 · 4 days ago History

Code Blame Executable File · 110 lines (92 loc) · 2.88 KB Raw

```
# Maxwell Heefner
# ECE209AS Trustworthy AI
# 18 Feb 26
# run_table1.sh
#!/usr/bin/env bash
set -euo pipefail

if [[ $# -lt 2 ]]; then
  echo "Usage: $0 <model_path> <coco_val2014_dir> [seed]"
  exit 1
fi

# All parameters taken from original project
MODEL_PATH="$1"
COCO_VAL_DIR="$2"
SEED="${3:-55}"

ROOT_DIR="${ROOT_DIR:-/content/VCD_project}"
ORIG_DIR="$ROOT_DIR/originalProject"
OUT_DIR="$ROOT_DIR/project1/output"
PYTHON_BIN="${PYTHON_BIN:-python3}"

CD_ALPHA="${CD_ALPHA:-1.0}"
CD_BETA="${CD_BETA:-0.2}"
NOISE_STEP="${NOISE_STEP:-500}"

mkdir -p "$OUT_DIR"

# Verify if the originalProject is available
```

```
if [[ ! -d "$ORIG_DIR/experiments" ]]; then
  echo "Missing experiments directory: $ORIG_DIR/experiments"
  exit 1
fi

if [[ ! -d "$COCO_VAL_DIR" ]]; then
  echo "Missing COCO val2014 directory: $COCO_VAL_DIR"
  exit 1
fi

# Run all POPE types (random and popular)
for split in random popular; do
  question_file="$ORIG_DIR/experiments/data/POPE/coco/coco_pope_${split}.json"

  # Run all method types (regular and vcd)
  for method in regular vcd; do
    answers_file="$OUT_DIR/llava15_coco_pope_${split}_answers_${method}_seed${SEE
    metrics_file="$OUT_DIR/metrics_${split}_${method}_seed${SEED}.txt"

    echo "Running split=$split method=$method"

    # .py script taken from original experiment
    cmd=(
      "$PYTHON_BIN" ./eval/object_hallucination_vqa_llava.py
      --model-path "$MODEL_PATH"
      --question-file "$question_file"
      --image-folder "$COCO_VAL_DIR"
      --answers-file "$answers_file"
      --seed "$SEED"
    )

    # Setup for VCD
    if [[ "$method" == "vcd" ]]; then
      cmd+=(
        --use_cd
        --cd_alpha "$CD_ALPHA"
        --cd_beta "$CD_BETA"
        --noise_step "$NOISE_STEP"
      )
    fi

    (
      cd "$ORIG_DIR/experiments"
```

```
          "${cmd[@]}"
          "$PYTHON_BIN" ./eval/eval_pope.py --gt_files "$question_file" --gen_files "
        )

        echo "Saved: $answers_file"
        echo "Saved: $metrics_file"
      done
    done

    # Return all outputs and format into a Table
    "$PYTHON_BIN" - <<'PY' "$OUT_DIR" "$SEED"
    import re
    import sys
    from pathlib import Path

    out_dir = Path(sys.argv[1])
    seed = sys.argv[2]

    def parse_metrics(path: Path):
        text = path.read_text()
        def g(name):
            m = re.search(rf"^{name}:\s*([0-9.]+)", text, re.MULTILINE)
            return float(m.group(1)) if m else float('nan')
        return {
            'accuracy': g('Accuracy'),
            'precision': g('Precision'),
            'recall': g('Recall'),
            'f1': g('F1'),
        }

    print("| Split | Method | Accuracy | Precision | Recall | F1 |")
    print("|---|---|---:|---:|---:|---:|")
    for split in ['random', 'popular']:
        for method in ['regular', 'vcd']:
            p = out_dir / f"metrics_{split}_{method}_seed{seed}.txt"
            m = parse_metrics(p)
            print(f"| {split} | {method} | {m['accuracy']:.4f} | {m['precision']:.4f}
    PY
```

Maxwell Heefner

ECE209AS Trustworthy AI

18 Feb 26

Project1 Task A

Uses code from original VCD Project, to re-create table 1 of: Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding

In [1]:

```
import torch
print('Torch:', torch.__version__)
print('CUDA:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
```

```
Torch: 2.9.0+cu128
CUDA: True
GPU: NVIDIA H100 80GB HBM3
```

In [2]:

```
%%bash
set -euo pipefail
cd /content
if [ ! -d VCD_project/.git ]; then
  git clone --depth 1 https://github.com/maxheef/ECE209_Project1.git VCD_pr
fi
cd /content/VCD_project
bash project1/setup_h100_env.sh /content/VCD_project
```

```
PREFIX=/usr/local/miniconda
Unpacking bootstrapper...
Unpacking payload...

Installing base environment...

Preparing transaction: ...working... done
Executing transaction: ...working... done
installation finished.
WARNING:
    You currently have a PYTHONPATH environment variable set. This may cause
```

Page 2-22 of Project1.ipnyb include the output of downloading the proper configurations/dependencies for Python 3.9, Numpy 1.X and PyTorch

These pages were omitted for brevity. To view the full report visit:
https://github.com/maxheef/ECE209_Project1.git

```
      Successfully uninstalled torchvision-0.15.2

Successfully installed nvidia-cublas-cu12-12.1.3.1 nvidia-cuda-cupti-cu12-1
2.1.105 nvidia-cuda-nvrtc-cu12-12.1.105 nvidia-cuda-runtime-cu12-12.1.105 nv
idia-cudnn-cu12-8.9.2.26 nvidia-cufft-cu12-11.0.2.54 nvidia-curand-cu12-10.
3.2.106 nvidia-cusolver-cu12-11.4.5.107 nvidia-cusparse-cu12-12.1.0.106 nvid
ia-nccl-cu12-2.19.3 nvidia-nvjitlink-cu12-12.9.86 nvidia-nvtx-cu12-12.1.105
torch-2.2.2+cu121 torchvision-0.17.2+cu121 triton-2.2.0
Collecting numpy<2
  Downloading numpy-1.26.4-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86
_64.whl.metadata (61 kB)
Downloading numpy-1.26.4-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_6
4.whl (18.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.2/18.2 MB 166.2 MB/s  0:00:00
Installing collected packages: numpy
  Attempting uninstall: numpy
    Found existing installation: numpy 2.0.2
    Uninstalling numpy-2.0.2:
      Successfully uninstalled numpy-2.0.2
Successfully installed numpy-1.26.4
Torch: 2.2.2+cu121
NumPy: 1.26.4
CUDA: True
GPU: NVIDIA H100 80GB HBM3
Cloning into 'VCD_project'...


==> WARNING: A newer version of conda exists. <==
    current version: 25.11.1
    latest version: 26.1.0

Please update conda by running

    $ conda update -n base -c defaults conda
```

In [3]:

```
%%bash
set -euo pipefail
mkdir -p /content/datasets/coco
cd /content/datasets/coco
if [ ! -d val2014 ]; then
  wget -q http://images.cocodataset.org/zips/val2014.zip
  unzip -q val2014.zip
fi
ls -la /content/datasets/coco/val2014 | sed -n '1,8p'
```

```
total 6564360
drwxrwxr-x 2 root root 2322432 Aug 16  2014 .
drwxr-xr-x 3 root root    4096 Feb 17 23:08 ..
-rw-rw-r-- 1 root root  213308 Aug 16  2014 COCO_val2014_000000000042.jpg
-rw-rw-r-- 1 root root  383651 Aug 16  2014 COCO_val2014_000000000073.jpg
-rw-rw-r-- 1 root root  176151 Aug 16  2014 COCO_val2014_000000000074.jpg
-rw-rw-r-- 1 root root  163607 Aug 16  2014 COCO_val2014_000000000133.jpg
-rw-rw-r-- 1 root root  105082 Aug 16  2014 COCO_val2014_000000000136.jpg
```

In [ ]:

```
%%bash
set -euo pipefail
cd /content/VCD_project
PYBIN=$(cat /tmp/project1_python_bin.txt)
PYTHON_BIN="$PYBIN" bash project1/run_table1.sh \
  liuhaotian/llava-v1.5-7b \
  /content/datasets/coco/val2014 \
  55
```

```
Running split=random method=regular
Precision: 0.9200680272108843
Recall: 0.7213333333333334
F1: 0.8086696562032885
Accuracy: 0.8293333333333334
yes: 0.392
unknow: 0.0
Saved: /content/VCD_project/project1/output/llava15_coco_pope_random_answers
_regular_seed55.jsonl
Saved: /content/VCD_project/project1/output/metrics_random_regular_seed55.tx
t
Running split=random method=vcd
Precision: 0.9368852459016394
Recall: 0.762
F1: 0.8404411764705884
Accuracy: 0.8553333333333333
yes: 0.4066666666666667
unknow: 0.0
Saved: /content/VCD_project/project1/output/llava15_coco_pope_random_answers
_vcd_seed55.jsonl
Saved: /content/VCD_project/project1/output/metrics_random_vcd_seed55.txt
Running split=popular method=regular
Precision: 0.8796747967479674
Recall: 0.7213333333333334
F1: 0.7926739926739926
Accuracy: 0.8113333333333334
yes: 0.41
unknow: 0.0
Saved: /content/VCD_project/project1/output/llava15_coco_pope_popular_answer
s_regular_seed55.jsonl
Saved: /content/VCD_project/project1/output/metrics_popular_regular_seed55.t
xt
Running split=popular method=vcd
Precision: 0.8943661971830986
Recall: 0.762
F1: 0.8228941684665229
Accuracy: 0.836
yes: 0.426
unknow: 0.0
Saved: /content/VCD_project/project1/output/llava15_coco_pope_popular_answer
s_vcd_seed55.jsonl
Saved: /content/VCD_project/project1/output/metrics_popular_vcd_seed55.txt
| Split | Method | Accuracy | Precision | Recall | F1 |
|---|---|---:|---:|---:|---:|
| random | regular | 0.8293 | 0.9201 | 0.7213 | 0.8087 |
| random | vcd | 0.8553 | 0.9369 | 0.7620 | 0.8404 |
| popular | regular | 0.8113 | 0.8797 | 0.7213 | 0.7927 |
| popular | vcd | 0.8360 | 0.8944 | 0.7620 | 0.8229 |
```

```
/usr/local/miniconda/envs/vcd39/lib/python3.9/site-packages/huggingface_hub/
file_download.py:949: FutureWarning: `resume_download` is deprecated and wil
l be removed in version 1.0.0. Downloads always resume when possible. If you
want to force a new download, use `force_download=True`.
  warnings.warn(
Downloading shards: 100%|██████████| 2/2 [00:30<00:00, 15.43s/it]
Loading checkpoint shards: 100%|██████████| 2/2 [00:08<00:00,  4.24s/it]
100%|██████████| 3000/3000 [03:37<00:00, 13.81it/s]
/usr/local/miniconda/envs/vcd39/lib/python3.9/site-packages/huggingface_hub/
file_download.py:949: FutureWarning: `resume_download` is deprecated and wil
l be removed in version 1.0.0. Downloads always resume when possible. If you
want to force a new download, use `force_download=True`.
  warnings.warn(
Loading checkpoint shards: 100%|██████████| 2/2 [00:08<00:00,  4.35s/it]
100%|██████████| 3000/3000 [06:26<00:00,  7.76it/s]
/usr/local/miniconda/envs/vcd39/lib/python3.9/site-packages/huggingface_hub/
file_download.py:949: FutureWarning: `resume_download` is deprecated and wil
l be removed in version 1.0.0. Downloads always resume when possible. If you
want to force a new download, use `force_download=True`.
  warnings.warn(
Loading checkpoint shards: 100%|██████████| 2/2 [00:08<00:00,  4.15s/it]
100%|██████████| 3000/3000 [03:32<00:00, 14.13it/s]
/usr/local/miniconda/envs/vcd39/lib/python3.9/site-packages/huggingface_hub/
file_download.py:949: FutureWarning: `resume_download` is deprecated and wil
l be removed in version 1.0.0. Downloads always resume when possible. If you
want to force a new download, use `force_download=True`.
  warnings.warn(
Loading checkpoint shards: 100%|██████████| 2/2 [00:08<00:00,  4.16s/it]
100%|██████████| 3000/3000 [06:23<00:00,  7.83it/s]
```

In [ ]:

```
# Verify that all files were created: Popular/Random x Regular/VCD and answ
%%bash
set -euo pipefail
ls -la /content/VCD_project/project1/output
```

```
total 2020
drwxr-xr-x 2 root root   4096 Feb 17 23:30 .
drwxr-xr-x 3 root root   4096 Feb 17 23:09 ..
-rw-r--r-- 1 root root 504920 Feb 17 23:24 llava15_coco_pope_popular_answers
_regular_seed55.jsonl
-rw-r--r-- 1 root root 504968 Feb 17 23:30 llava15_coco_pope_popular_answers
_vcd_seed55.jsonl
-rw-r--r-- 1 root root 505842 Feb 17 23:13 llava15_coco_pope_random_answers_
regular_seed55.jsonl
-rw-r--r-- 1 root root 505886 Feb 17 23:20 llava15_coco_pope_random_answers_
vcd_seed55.jsonl
-rw-r--r-- 1 root root    131 Feb 17 23:24 metrics_popular_regular_seed55.tx
t
-rw-r--r-- 1 root root    106 Feb 17 23:30 metrics_popular_vcd_seed55.txt
-rw-r--r-- 1 root root    132 Feb 17 23:13 metrics_random_regular_seed55.txt
-rw-r--r-- 1 root root    132 Feb 17 23:20 metrics_random_vcd_seed55.txt
```